

“Año de la Esperanza y el Fortalecimiento de la Democracia”

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad del Perú, Decana de América

FACULTAD DE INGENIERÍA ELECTRÓNICA Y ELÉCTRICA

E.P. de Ingeniería de Telecomunicaciones



Entregable 4: Sistema Completo Verificado y Borrador de Paper

Diseño e implementación de un modulador FSK digital basado en Arduino con selección de secuencia binaria mediante DIP Switch y medición de frecuencia usando un segundo Arduino

DOCENTE:

Villafuerte Barreto, Hernán Oswaldo

CURSO:

Circuitos Digitales

INTEGRANTES:

Farro Pomajulca Leonardo Jose

Cruz Vargas José Antonio

Bellido Araujo Alvaro Daniel

Lima, 2026.

Entregable 4

1. Evidencia de Funcionalidades Operativas

En esta sección se documenta, mediante capturas del simulador Tinkercad, el funcionamiento completo del sistema: la implementación del circuito, la transmisión de cuatro combinaciones representativas del DIP switch, la verificación del manejo de ausencia de señal, y la respuesta del filtro RC observada en el osciloscopio virtual.

1.1. Circuito implementado

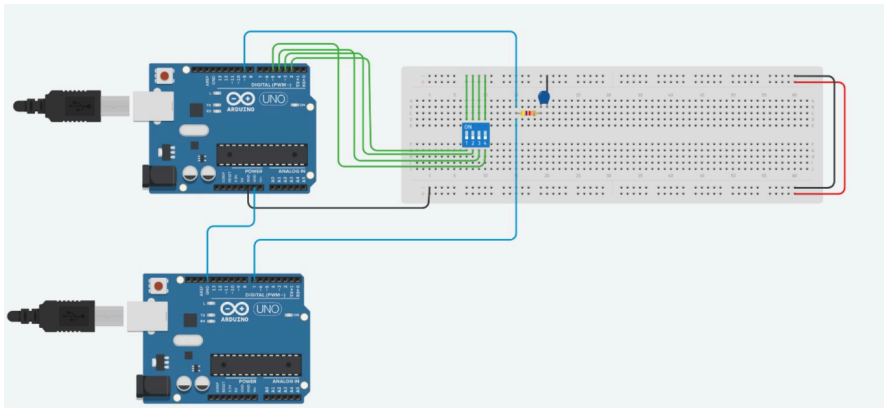


Fig. 1. Circuito implementado en Tinkercad: Arduino 1 (transmisor), DIP switch de 4 posiciones, filtro RC y Arduino 2 (medidor).

1.2. Transmisión y medición — Combinación DIP = 1,0,1,0

El Arduino 1 transmitió la secuencia esperada, alternando correctamente entre ambas frecuencias, mientras que el Arduino 2 midió una frecuencia cercana al valor teórico para cada bit.

Arduino 1 — Transmisor	Arduino 2 — Medidor
Bit transmitido 1: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2044.99 Hz
Bit transmitido 2: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1131.22 Hz
Bit transmitido 3: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2136.75 Hz
Bit transmitido 4: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1144.16 Hz
---- Reinicia secuencia ----	

Fig. 2. Monitores seriales para la combinación 1,0,1,0.

1.3. Transmisión y medición — Combinación DIP = 1,1,1,1

Arduino 1 — Transmisor	Arduino 2 — Medidor
Bit transmitido 1: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2083.33 Hz
Bit transmitido 2: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2087.68 Hz
Bit transmitido 3: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2136.75 Hz
Bit transmitido 4: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2087.68 Hz
---- Reinicia secuencia ----	

Fig. 3. Monitores seriales para la combinación 1,1,1,1 (caso sin flancos, bit 1 constante).

1.4. Transmisión y medición — Combinación DIP = 0,0,0,0

Arduino 1 — Transmisor	Arduino 2 — Medidor
Bit transmitido 1: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1128.67 Hz
Bit transmitido 2: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1141.55 Hz
Bit transmitido 3: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1161.44 Hz
Bit transmitido 4: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1131.22 Hz

Fig. 4. Monitores seriales para la combinación 0,0,0,0 (caso sin flancos, bit 0 constante).

1.5. Transmisión y medición — Combinación DIP = 0,1,1,0

Arduino 1 — Transmisor	Arduino 2 — Medidor
Bit transmitido 1: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1131.22 Hz
Bit transmitido 2: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2092.05 Hz
Bit transmitido 3: 1 Frecuencia: 2200 Hz	MEDIDOR Frecuencia medida: 2083.33 Hz
Bit transmitido 4: 0 Frecuencia: 1200 Hz	MEDIDOR Frecuencia medida: 1164.14 Hz
---- Reinicia secuencia ----	

Fig. 5. Monitores seriales para la combinación 0,1,1,0 (flanco único a mitad de trama).

1.6. Verificación del manejo de ausencia de señal

Con la combinación DIP = 1,1,1,1 transmitiendo normalmente, se desconectó físicamente el enlace D9 (Arduino 1) → D7 (Arduino 2) dentro del simulador. El Arduino 2 identificó correctamente la ausencia de señal en las cuatro iteraciones consecutivas, validando la rama de manejo de error basada en el tiempo de espera (timeout) de pulseIn().

Arduino 1 — Sigue transmitiendo	Arduino 2 — Enlace desconectado
Bit transmitido 1: 1 Frecuencia: 2200 Hz	MEDIDOR Sin señal detectada
Bit transmitido 2: 1 Frecuencia: 2200 Hz	MEDIDOR Sin señal detectada
Bit transmitido 3: 1 Frecuencia: 2200 Hz	MEDIDOR Sin señal detectada
Bit transmitido 4: 1 Frecuencia: 2200 Hz	MEDIDOR Sin señal detectada

Fig. 6. Verificación de la detección de ausencia de señal (D9–D7 desconectado).

1.7. Respuesta del filtro RC — Osciloscopio virtual

Se capturó la forma de onda antes y después del filtro RC para ambas frecuencias de operación, utilizando la misma escala temporal (1000 μs) para permitir una comparación directa.

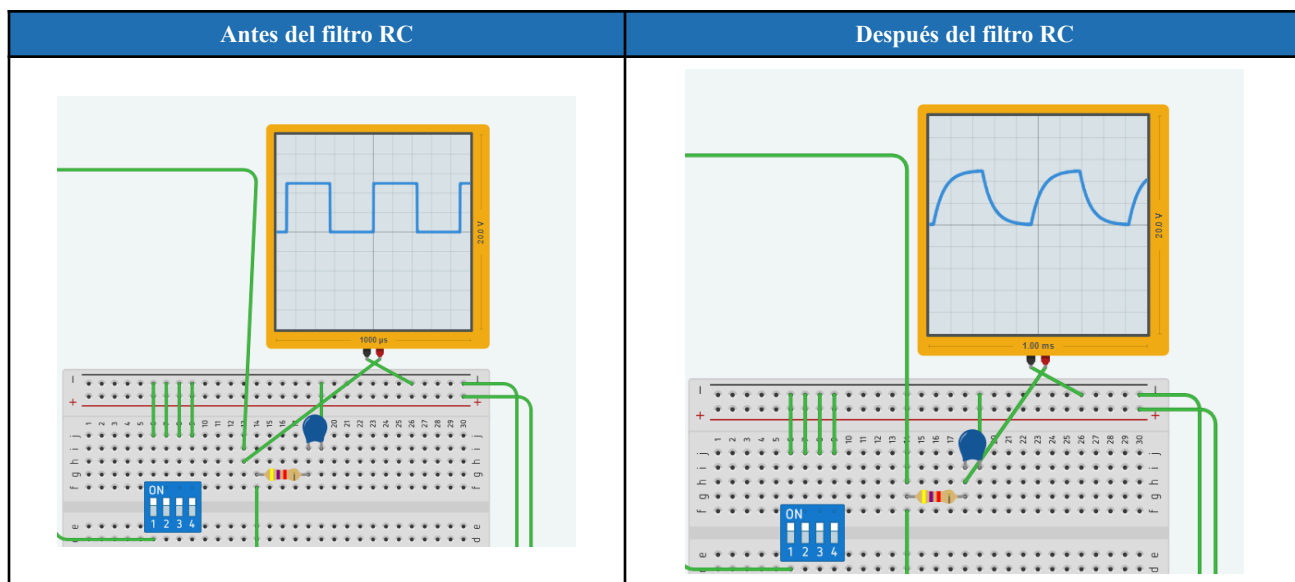


Fig. 7. Forma de onda a 2200 Hz (bit = 1, combinación 1,1,1,1) antes y después del filtro RC.

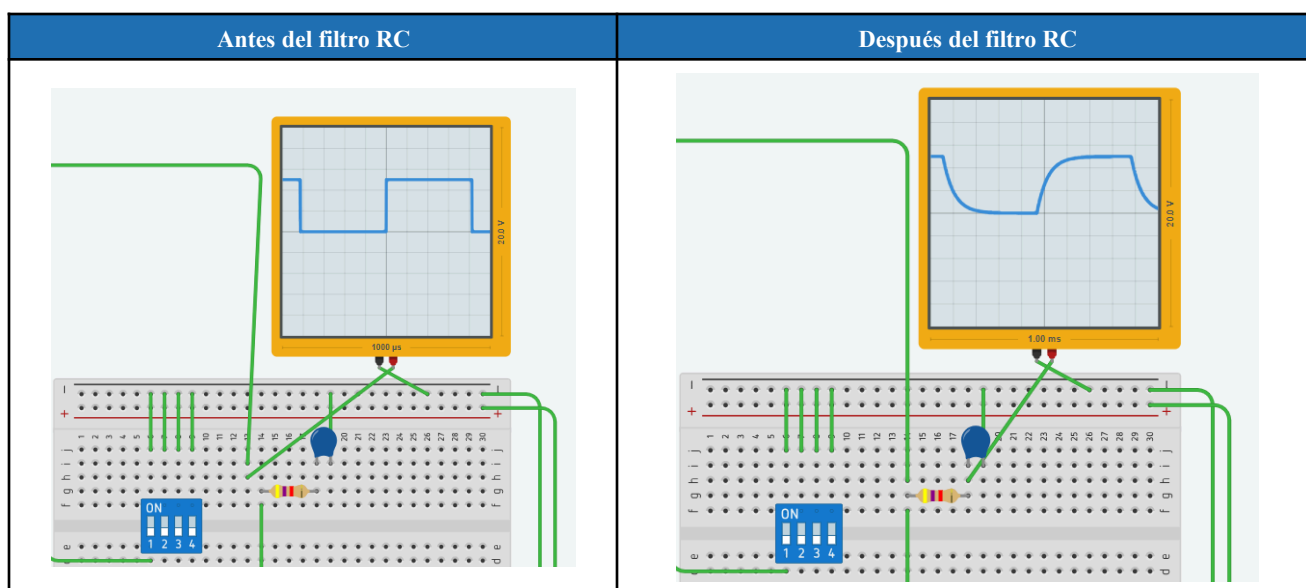


Fig. 8. Forma de onda a 1200 Hz (bit = 0, combinación 0,0,0,0) antes y después del filtro RC.

En ambos casos se observa el mismo efecto: la onda cuadrada original presenta transiciones abruptas entre 0 y 5 V, mientras que la señal posterior al filtro muestra flancos redondeados, aproximándose a una forma semi-senoidal sin llegar a serlo por completo. Esto confirma cualitativamente el comportamiento esperado del filtro RC de primer orden como suavizador parcial de la señal.

2. Medición de Métricas

Métrica 1 — Precisión de frecuencia (teórico vs. simulado)

Se registraron 16 mediciones de frecuencia (4 combinaciones del DIP switch $\times$ 4 bits cada una), comparando el valor teórico contra el valor reportado por el Arduino 2 en Tinkercad.

TABLA I. FRECUENCIA TEÓRICA VS. SIMULADA

DIP	Bit#	Bit	F. teórica (Hz)	F. simulada (Hz)	Error %
1,0,1,0	1	1	2200	2044.99	7.05%
1,0,1,0	2	0	1200	1131.22	5.73%
1,0,1,0	3	1	2200	2136.75	2.88%
1,0,1,0	4	0	1200	1144.16	4.65%
1,1,1,1	1	1	2200	2083.33	5.30%
1,1,1,1	2	1	2200	2087.68	5.11%
1,1,1,1	3	1	2200	2136.75	2.88%
1,1,1,1	4	1	2200	2087.68	5.11%
0,0,0,0	1	0	1200	1128.67	5.94%
0,0,0,0	2	0	1200	1141.55	4.87%
0,0,0,0	3	0	1200	1161.44	3.21%
0,0,0,0	4	0	1200	1131.22	5.73%
0,1,1,0	1	0	1200	1131.22	5.73%
0,1,1,0	2	1	2200	2092.05	4.91%
0,1,1,0	3	1	2200	2083.33	5.30%
0,1,1,0	4	0	1200	1164.14	2.99%

Resumen estadístico: frecuencia de 2200 Hz — promedio medido 2094.07 Hz (error promedio 4.82%); frecuencia de 1200 Hz — promedio medido 1141.70 Hz (error promedio 4.86%). Error promedio general: 4.84%. Error mínimo observado: 2.88%. Error máximo observado: 7.05%.

Métrica 2 — Duración de bit (limitación declarada)

La duración teórica de cada bit, definida por la constante `duracionBit = 1000 ms` en el firmware, no fue medida experimentalmente en este entregable, ya que no se instrumentó el código con marcas de tiempo (`millis()`) en los mensajes seriales. Se declara esta limitación explícitamente: el valor reportado en este documento (1000 ms) corresponde únicamente al valor teórico de diseño. Se recomienda como trabajo futuro instrumentar el firmware para medir este parámetro de forma experimental.

Métrica 3 — Respuesta del filtro RC (evidencia cualitativa)

A partir de las Fig. 7 y Fig. 8, se verifica cualitativamente que el filtro RC suaviza los flancos de la señal cuadrada en ambas frecuencias de operación (1200 Hz y 2200 Hz), sin distorsionar la forma de onda hasta el punto de perder la información de

frecuencia. No se dispuso de un método directo en Tinkercad para cuantificar el tiempo de subida (rise time) o la atenuación en amplitud, por lo que esta métrica se reporta de forma cualitativa (presente/ausente el suavizado), no numérica.

Métrica 4 — Tasa de identificación correcta

Tomando como criterio de aceptación una tolerancia de $\pm 10\%$ respecto del valor teórico (margen razonable para un sistema basado en temporizadores de software y funciones estándar de Arduino), las 16 mediciones de frecuencia de la Tabla I caen dentro de dicho margen, resultando en una tasa de identificación correcta de 16/16 (100%). Asimismo, la prueba de ausencia de señal (Fig. 6) fue identificada correctamente en las 4 iteraciones observadas (4/4, 100%).

TABLA II. RESUMEN DE LAS 4 MÉTRICAS EVALUADAS

#	Métrica	Resultado	Tipo de dato
1	Precisión de frecuencia	Error promedio 4.84% (rango 2.88%–7.05%)	Cuantitativo (teórico vs. simulado)
2	Duración de bit	1000 ms (solo valor teórico)	No medido — limitación declarada
3	Respuesta del filtro RC	Suavizado de flancos confirmado en 1200 Hz y 2200 Hz	Cualitativo (evidencia visual)
4	Tasa de identificación correcta	16/16 frecuencias ($\pm 10\%$); 4/4 detección de ausencia de señal	Cuantitativo (simulado)

3. Análisis de Desviaciones y Posibles Causas

El hallazgo más relevante de la Tabla I es que las 16 mediciones de frecuencia son sistemáticamente menores al valor teórico, nunca mayores, y con un error promedio muy similar entre ambas frecuencias (4.82% para 2200 Hz vs. 4.86% para 1200 Hz). Esta consistencia direccional (siempre por debajo) descarta que se trate de ruido aleatorio de medición y apunta a una fuente de error sistemática. Se identifican las siguientes causas probables:

- Sobrecarga de tiempo de ejecución de `pulseIn()`: esta función bloquea el procesador mientras espera flancos de la señal; el tiempo que tarda en iniciar la captura y en procesar el resultado se suma al periodo medido, alargándolo artificialmente y, por lo tanto, reduciendo la frecuencia calculada ($f = 1/T$).
- Resolución del temporizador de `tone()`: la función `tone()` genera la frecuencia dividiendo la base de tiempo de 16 MHz del microcontrolador entre un entero; para ciertas frecuencias el divisor no es exacto, generando una frecuencia real ligeramente distinta a la nominal, incluso antes de que el Arduino 2 la mida.

- Efecto de carga del filtro RC: el filtro RC introduce un pequeño retardo de fase y un tiempo de carga/descarga del capacitor, lo cual puede desplazar el instante exacto en que la señal cruza el umbral lógico que `pulseIn()` utiliza para detectar el flanco, sesgando el periodo medido.
- Diferencia en la variabilidad entre combinaciones: se observa que en la combinación 0,0,0,0 el error decrece progresivamente en las mediciones sucesivas (5.94% $\rightarrow$ 4.87% $\rightarrow$ 3.21% $\rightarrow$ 5.73%), lo que sugiere una fase de estabilización inicial del bucle de medición del Arduino 2, posiblemente asociada al primer ciclo tras el estado de 'sin señal detectada'.

En conjunto, estas causas son consistentes con las limitaciones conocidas de una medición de frecuencia por captura de pulsos en un microcontrolador de propósito general, y no evidencian un error de diseño en la lógica del decodificador o de la máquina de estados, cuyo comportamiento lógico (secuencia de bits y selección de frecuencia) fue en todos los casos el esperado.

IV. CONCLUSIONES

Los resultados preliminares confirman que el sistema modulador FSK diseñado reproduce correctamente, a nivel lógico, la secuencia de frecuencias esperada para cualquier combinación del DIP switch, y que el mecanismo de detección de ausencia de señal opera de forma confiable. Sin embargo, se observa una desviación sistemática de aproximadamente 4.8% entre la frecuencia teórica y la medida en simulación, atribuible principalmente al tiempo de ejecución de `pulseIn()` y a la resolución del temporizador de `tone()`, más que a un error de diseño lógico.

Como trabajo futuro se plantea: (i) instrumentar el firmware con marcas de tiempo (`millis()`) para medir experimentalmente la duración real de cada bit; (ii) validar el sistema sobre hardware físico, empleando un osciloscopio o multímetro real, para contrastar la desviación aquí reportada frente a un montaje real; y (iii) evaluar filtros RC con distintos valores de R y C para cuantificar su efecto sobre la fidelidad de la señal recibida.

REFERENCIAS

- [1] Arduino, "tone() – Language Reference", Arduino Reference, Arduino AG. Disponible en: <https://www.arduino.cc/reference/en/language/functions/advanced-io/tone/>
 - [2] Arduino, "pulseIn() – Language Reference", Arduino Reference, Arduino AG. Disponible en: <https://www.arduino.cc/reference/en/language/functions/advanced-io/pulsein/>
 - [3] Arduino, "millis() – Language Reference", Arduino Reference, Arduino AG. Disponible en: <https://www.arduino.cc/reference/en/language/functions/time/millis/>
 - [4] B. Sklar, Digital Communications: Fundamentals and Applications, 2nd ed. Upper Saddle River, NJ, USA: Prentice Hall, 2001.
 - [5] M. M. Mano y M. D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, 5th ed. Boston, MA, USA: Pearson, 2013.
 - [6] A. S. Sedra y K. C. Smith, Microelectronic Circuits, 7th ed. New York, NY, USA: Oxford University Press, 2015.
 - [7] Autodesk, "Tinkercad Circuits — Documentation", Autodesk Inc. Disponible en: <https://www.tinkercad.com/>
 - [8] Microchip Technology Inc., "ATmega328P — Datasheet", Microchip Technology Inc. Disponible en: <https://www.microchip.com/>
-